УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 12

СОЗДАНИЕ ТИПА ДАННЫХ «КЛАСС»

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание типа данных «класс»

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

Объектно-ориентированное программирование (ООП) – это способ организации программы, позволяющий использовать один и тот же код многократно. В отличие от функций и модулей, ООП позволяет не только разделить программу на фрагменты, но и описать предметы реального мира в виде удобных сущностей - **объектов**, а также организовать связи между этими объектами.

Основным «кирпичиком» **ООП** является класс. **Класс** – это сложный тип данных, включающий набор переменных и функций для управления значениями, хранящимися в этих переменных. Переменные называют **атрибутами**, а функции - **методами**. Класс является фабрикой объектов, т.е. позволяет создать неограниченное количество экземпляров, основанных на этом классе.

Разработка программного обеспечения средствами ООП имеет массу преимуществ, причем это относится не только к созданию более эффективного программного кода, но и к модификации и расширению возможностей уже имеющихся программных средств.

Два основных аспекта объектно-ориентированного программирования – классы и объекты. **Класс** создает новый тип, а **объекты** являются **экземплярами** класса. Аналогично, когда говорим о «переменных типа int», это означает, что переменные, которые хранят целочисленные значения, являются экземплярами (объектами) класса **int.**

Объекты могут хранить данные в обычных переменных, которые принадлежат объекту. Переменные, принадлежащие объекту или классу, называют **полями**. Объекты могут также обладать функционалом, т.е. иметь функции, принадлежащие классу. Такие функции в ООП принято называть **методами** класса. Эта терминология важна, так как она помогает отличать независимые

функции и переменные от тех, что принадлежат классу или объекту. Все вместе (поля и методы) принято называть **атрибутами** класса.

Поля бывают двух типов: они могут принадлежать каждому отдельному экземпляру объекта класса или всему классу. Они называются **переменными экземпляра** и **переменными класса**, соответственно.

4.1 Определение класса в Python

Класс описывается с помощью ключевого слова **class** по следующей схеме:

```
class <Название класса> [(<Класс1>[, ...,  <КлассN>])]:
   ["""  Строка документирования  """]
   <Описание  атрибутов и методов>
```

Инструкция создает новый объект и присваивает ссылку на него идентификатору, указанному после ключевого слова **class**. Это означает, что название класса должно полностью соответствовать правилам именования переменных. После названия класса в круглых скобках можно указать один или несколько базовых классов через запятую. Если же класс не наследует базовые классы, то круглые скобки можно не указывать. Следует заметить, что все выражения внутри инструкции **class** выполняются при создании класса, а не его экземпляра. Для примера создадим класс, внутри которого просто выводится сообщение:

```
# Создание определения класса
# -*- coding: utf-8 -*-
class MyClass:
   """ Это строка документирования """
   print("Инструкции выполняются сразу")
input()
```

Этот пример содержит лишь определение класса **MyClass** и не создает экземпляр класса. Как только поток выполнения достигнет инструкции **class**, сообщение, указанное в функции **print()**, будет сразу выведено.

Создание атрибута класса аналогично созданию обычной переменной. Метод внутри класса создается так же, как и обычная функция, - с помощью инструкции **def**. Методам класса в первом параметре, который обязательно следует указать явно, автоматически передается ссылка на экземпляр класса. Общепринято этот параметр называть именем **self**, хотя это и не обязательно. Доступ к атрибутам и методам класса внутри определяемого метода производится через переменную **self** с помощью точечной нотации - к атрибуту **x** из метода класса можно обратиться так: **self.x**.

4.2 Создание экземпляра класса в Python

Чтобы использовать атрибуты и методы класса, необходимо создать экземпляр класса согласно следующему синтаксису:

```
<Экземпляр класса> = <Название класса> ( [<Параметры>] )
```

При обращении к методам класса используется такой формат:

<Экземпляр класса>.<Имя метода> ([<Параметры>])

Обратите внимание на то, что при вызове метода не нужно передавать ссылку на экземпляр класса в качестве параметра, как это делается в определении метода внутри класса. Ссылку на экземпляр класса интерпретатор передает автоматически.

Обращение к атрибутам класса осуществляется аналогично:

<Экземпляр класса>.<Имя атрибута>

Определим класс **MyClass** с атрибутом **x** и методом **print_x ()**, выводящим значение этого атрибута, а затем создадим экземпляр класса и вызовем метод:

```
# Создание атрибута и метода
# -*- coding: utf-8 -*-
class MyClass:
    def __init__(self):       #  Конструктор
        self.x =10   #  Атрибут  экземпляра  класса
    def print_x(self):        #  self - это ссылка на экземпляр класса
        print(self.x)# Выводим значение атрибута
c = MyClass() # Создание экземпляра класса
#  Вызываем метод print_x()
c.print_x() #  self не указывается при  вызове метода
print(c.x)  #  К атрибуту можно обратиться непосредственно
Результат работы приложения:
10
10
```

Для доступа к атрибутам и методам можно использовать и следующие функции:

- **getattr()** - возвращает значение атрибута по его названию, заданному в виде строки. С помощью этой функции можно сформировать имя атрибута динамически во время выполнения программы. Формат функции:

  getattr(<Объект>, <Атрибут>[, <Значение по умолчанию>])

  Если указанный атрибут не найден, возбуждается исключение **AttributeError**. Чтобы избежать вывода сообщения об ошибке, можно в третьем параметре указать значение, которое будет возвращаться, если атрибут не существует;

- **setattr()** - задает значение атрибута. Название атрибута указывается в виде строки. Формат функции:

  setattr(<Объект>, <Атрибут>, <Значение>)

  Вторым параметром метода **setattr()** можно передать имя несуществующего атрибута - в этом случае атрибут с указанным именем будет создан;

- **delattr(<Объект>, <Атрибут>)** - удаляет указанный атрибут. Название атрибута указывается в виде строки;

- **hasattr(<Объект>, <Атрибут>)** - проверяет наличие указанного атрибута. Если атрибут существует, функция возвращает значение **True**.

Продемонстрируем работу функций на следующем примере:

```
class MyClass:
```

```
    def __init__(self):
        self.x = 10
    def get_x(self):
        return self.x
c = MyClass()               # Создаем экземпляр класса
print(getattr(c, "x"))      # Выведет: 10
print(getattr(c, "get_x" ) ()) # Выведет: 10
print(getattr(c, "y", 0))   # Выведет: 0, т. к. атрибут не найден
setattr(c, "y", 20)         # Создаем атрибут y
print(getattr(c, "y", 0))   # Выведет: 20
delattr(c, "y")             # Удаляем атрибут y
print(getattr(c, "y", 0))   # Выведет: 0, т. к. атрибут не найден
print(hasattr(c, "x") )     # Выведет: True
print(hasattr(c, "y") )     # Выведет: False
```

Результат работы приложения:

```
10
10
0
20
0
True
False
```

Все атрибуты класса в языке **Python** являются **открытыми (public)**, т. е. доступными для непосредственного изменения как из самого класса, так и из других классов, и из основного кода программы.

Кроме того, атрибуты допускается создавать динамически после создания класса - можно создать как атрибут объекта класса, так и атрибут экземпляра класса. Рассмотрим это на следующем примере:

```
class MyClass:              # Определяем пустой класс
    pass
MyClass.x = 50              # Создаем атрибут объекта класса
c1, c2 = MyClass(), MyClass() # Создаем два экземпляра класса
c1.y = 10                   # Создаем атрибут экземпляра класса
c2.y = 20                   # Создаем атрибут экземпляра класса
print(c1.x, c1.y)           # Выведет: 50 10
print(c2.x, c2.y)           # Выведет: 50 20
```

Результат работы приложения:

```
50 10
50 20
```

В этом примере определяем пустой класс, разместив в нем оператор **pass**. Далее создаем атрибут объекта класса (**x**). Этот атрибут будет доступен всем создаваемым экземплярам класса. Затем создаем два экземпляра класса и добавляем одноименные атрибуты (**y**). Значения этих атрибутов будут разными в каждом экземпляре класса. Но если создать новый экземпляр (например, **c3**), то атрибут y в нем определен не будет. Таким образом, с помощью классов можно

имитировать типы данных, поддерживаемые другими языками программирования (например, тип **struct**, доступный в языке **C**).

Очень важно понимать разницу между атрибутами объекта класса и атрибутами экземпляра класса. Атрибут объекта класса доступен всем экземплярам класса, но после изменения атрибута значение изменится во всех экземплярах класса. Атрибут экземпляра класса может хранить уникальное значение для каждого экземпляра, и изменение его в одном экземпляре класса не затронет значения одноименного атрибута в других экземплярах того же класса. Рассмотрим это на примере, создав класс с атрибутом объекта класса (**x**) и атрибутом экземпляра класса (**y**):

```
class MyClass:
    x = 10;              # Атрибут экземпляра класса
    def __init__(self):
        self.y = 20      # Атрибут экземпляра класса
c1 = MyClass()           # Создаем экземпляр класса
c2 = MyClass()           # Создаем экземпляр класса
print(c1.x, c2.x)        # Выведет: 10 10
MyClass.x = 88           # Изменяем атрибут объекта класса
print(c1.x, c2.x)        # Выведет: 88 88
print(c1.y, c2.y)        # Выведет: 20 20
c1.y = 88                # Изменяем атрибут экземпляра класса
print(c1.y, c2.y)        # Выведет: 88 20
```

Результат работы приложения:

```
10 10
88 88
20 20
88 20
```

Сначала создается класс **MyClass**, а затем два экземпляра этого класса:

```
c1 = MyClass()           # Создаем экземпляр класса
c2 = MyClass()           # Создаем экземпляр класса
```

Выведем значения атрибута **x**, а затем изменим значение и опять произведем вывод:

```
print(c1.x, c2.x)        # Выведет: 10 10
MyClass.x = 88           # Изменяем атрибут объекта класса
print(c1.x, c2.x)        # Выведет: 88 88
```

Как видно из примера, изменение атрибута объекта класса затронуло значение в двух экземплярах класса сразу. Теперь произведем аналогичную операцию с атрибутом **y**:

```
print(c1.y, c2.y)        # Выведет: 20 20
c1.y = 88                # Изменяем атрибут экземпляра класса
print(c1.y, c2.y)        # Выведет: 88 20
```

В этом случае изменилось значение атрибута только в экземпляре **c1**.

Следует также учитывать, что в одном классе могут одновременно существовать атрибут объекта и атрибут экземпляра с одним именем. Изменение атрибута объекта класса производили следующим образом:

```
MyClass.x = 88          # Изменяем атрибут объекта класса
```

Если после этой инструкции вставить инструкцию

```
c1.x = 200              # Создаем атрибут экземпляра
```

то будет создан атрибут экземпляра класса, а не изменено значение атрибута объекта класса. Чтобы увидеть разницу, выведем значения атрибута:

```
print(c1.x, MyClass.x)    # Выведет: 200 88
```

4.3 Методы __init__() и __del__()

При создании экземпляра класса интерпретатор автоматически вызывает метод инициализации **__init__()**. В других языках программирования такой метод принято называть **конструктором класса**. Формат метода:

```
def __init__(self [, <Значение1>[, ..., <ЗначениеN>]]):
    <Инструкции>
```

С помощью метода **__init__()** можно присвоить начальные значения атрибутам класса.

При создании экземпляра класса параметры этого метода указываются после имени класса в круглых скобках:

```
<Экземпляр класса>  = <Имя  класса>([<Значение1>[, ..., <ЗначениеN>]])
```

Пример использования метода **__init__()**:

```
class MyClass:
    def __init__(self, value1, value2):  # Конструктор
        self.x = value1
        self.y = value2
c = MyClass(100, 300)                 # Создаем экземпляр класса
print(c.x, c.y)                       # Выведет: 100 300
```

Результат работы приложения:

```
100 300
```

Если конструктор вызывается при создании объекта, то перед уничтожением объекта автоматически вызывается метод, называемый **деструктором**. В языке **Python** деструктор реализуется в виде предопределенного метода **__del__()**. Следует заметить, что метод не будет вызван, если на экземпляр класса существует хотя бы одна ссылка. Впрочем, поскольку интерпретатор самостоятельно заботится об удалении объектов, использование деструктора в языке **Python** не имеет особого смысла.

Пример использования деструктора:

```
class MyClass:
    def  __init__(self):   # Конструктор класса
        print ( "Вызван метод __init__()")
    def __del__(self):     # Деструктор класса
        print ( "Вызван метод __del__()")

c1 = MyClass() # Выведет: Вызван метод        __init__()
del c1        # Выведет: Вызван метод    __del__()
c2 = MyClass() # Выведет: Вызван метод        __init__()
```

```
c3 = c2          # Создаем ссылку на экземпляр класса
del c2           # Ничего не выведет, т. к. существует ссылка
del c3           # Выведет: Вызван метод __del__()
```

Результат работы приложения:

```
Вызван метод __init__()
Вызван метод __del__()
Вызван метод __init__()
Вызван метод __del__()
```

4.4 Пример решения задачи на языке программирования Python

Задача. Создайте класс с методом-конструктором, в котором следует определить атрибуты экземпляра класса, необходимые для сложения двух целых чисел. Напишите метод, в котором бы определялась сумма двух целых чисел.

Комментарий. Как уже известно, с помощью метода **__init__** можно присвоить начальные значения атрибутам класса. Ниже строками кода **self.a=chislo1** и **self.b=chislo2** создаем атрибуты **a, b** у объекта и присваиваем им переменные **chislo1**, **chislo2**. В созданном далее методе **summ** обращаемся к атрибутам **a, b** как **self.a** и **self.b**. Чтобы использовать атрибуты и метод класса, создаем экземпляр класса **rez**, в котором значения первого и второго числа указываются после имени класса **Summa** путем запроса с помощью функции **input**. Вызов метода **summ()** завершает программу.

```
class Summa():
    def __init__(self, chislo1, chislo2):
        self.a=chislo1
        self.b=chislo2
    def summ(self):
        x=self.a+self.b
        print('Сумма чисел = {0}'.format(x))
rez=Summa(int(input('Введите первое число:')),int(input('Введите второе
число:')))
rez.summ()
```

Ниже представлен еще один код программы, условие которой изложено в задаче. Изменения произошли в методе summ() за счет использования оператора return, который будет возвращать значение x, полученное от сложения двух чисел.

```
class Summa():
    def __init__(self, chislo1, chislo2):
        self.a=chislo1
        self.b=chislo2
    def summ(self):
        x=self.a+self.b
        return x
rez=Summa(int(input('Введите первое число:')),int(input('Введите второе
число:')))
print(rez.summ())
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Создайте класс «Пирамида», содержащий поля для хранения необходимых размеров пирамиды. Реализуйте методы вычисления объема и площади боковой поверхности. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

2 Создайте класс «Конус», содержащий поля для хранения необходимых размеров конуса. Реализуйте методы вычисления объема и площади боковой поверхности. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

3 Создайте класс «Шар», содержащий поля для хранения радиуса шара. Реализуйте методы вычисления объема и площади поверхности. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

4 Создайте класс «Призма», содержащий поля для хранения необходимых размеров призмы. Реализуйте методы вычисления объема и площади боковой поверхности. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

5 Создайте класс «Цилиндр», содержащий поля для хранения радиуса основания и высоты. Реализуйте методы вычисления объема и площади боковой поверхности. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

6 Создайте класс «Круг», содержащий поле для хранения радиуса. Реализуйте методы вычисления площади и длины окружности. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

7 Создайте класс «Треугольник», содержащий поля для хранения длин сторон. Реализуйте методы вычисления площади и периметра. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

8 Создайте класс «Куб», содержащий поле для хранения длины ребра. Реализуйте методы вычисления объема и площади поверхности. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

9 Создайте класс «Прямоугольник», содержащий поля для хранения длины и ширины. Реализуйте методы вычисления площади и периметра. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

10 Создайте класс «Параллелепипед», содержащий поля для хранения длины, ширины и высоты. Реализуйте методы вычисления объема и площади поверхности. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

11 Создайте класс «Эллипс», содержащий поля для хранения большой и малой полуосей. Реализуйте методы вычисления площади и длины эллипса. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

12 Создайте класс «Квадрат», содержащий поле для хранения длины стороны. Реализуйте методы вычисления площади и периметра. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

13 Создайте класс «Параллелограмм», содержащий поля для хранения сторон и высоты. Реализуйте методы вычисления площади и периметра. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

14 Создайте класс «Трапеция», содержащий поля для хранения длин оснований, боковых сторон и высоты. Реализуйте методы вычисления площади и периметра. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

15 Создайте класс «Ромб», содержащий поля для хранения длины стороны и диагоналей. Реализуйте методы вычисления площади и периметра. Создайте экземпляр класса, вызовите методы и выведите результаты вычислений на экран.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы

6.2 Цель лабораторной работы

6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Дайте определение понятию «Класс» в языке программирования Python.

7.2 Опишите синтаксис создания класса в языке программирования Python.

7.3 Опишите синтаксис создания экземпляра класса в языке программирования Python.

7.4 Охарактеризуйте методы __init__() и __del__() в языке программирования Python.

7.5 Опишите функции для доступа к атрибутам и методам класса в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.